

プログラミング応用 期末課題解説

問 1 グラフ $G = (V, E)$ がグラフ $H = (W, F)$ を含むとは, ある単射 $\phi: W \rightarrow V$ が存在して, 全ての H の辺 $\{a, b\} \in F$ に対して $\{\phi(a), \phi(b)\} \in E$ を満たすことを言う. 隣接行列として与えられた n 頂点グラフ G が次の図 1 で表されるグラフ H を含むかどうかを判定する問題を考える.

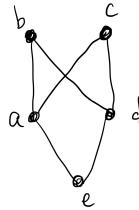


図 1: グラフ H

1. この問題を $O(n^3)$ 時間で判定するアルゴリズムを記述せよ. (4 点)
2. この問題を $O(n^2)$ 時間で判定するアルゴリズムを記述せよ. (6 点)

【解答】

1 の解説. 隣接行列 A に対し, A^2 を $O(n^3)$ 時間で計算し, $A^2_{u,v} \geq 3$ を満たす $u \neq v$ が存在するかどうかを確認すればよい.

採点基準. アルゴリズムが合ってるかどうか. 点数は 0, 2, 4 の 3 段階.

2 の解説. 与えられたグラフを $G = (V, E)$ とする. 頂点 u の隣接頂点の集合を $N(u)$ とする. 以下のアルゴリズムを考える:

1. 二次元 $n \times n$ リスト C を用意し, 全ての成分を 0 で初期化する.
2. 全ての $u \in V$ に対して以下を実行する:
 - (a) 全ての $v \in N(u)$ に対して, $C_{u,v} \leftarrow C_{u,v} + 1$ とする.
 - (b) $C_{u,v} \geq 3$ ならば, 「Yes」を出力して終了.
3. 最後まで終了しなければ, 「No」を出力して終了.

このアルゴリズムは $O(n^2)$ 時間で動作する. 実際, もしも 2 の反復が $2n^2 + 1$ 回実行されてもなお終了していないとき, C の成分の個数は n^2 個なので, 必ずどこかの成分は 3 回以上 +1 されているはずであるが, その瞬間にアルゴリズムは終了するため, 矛盾する.

採点基準. アルゴリズムと解析が合ってるかどうか. 点数は 0, 3, 6 の 3 段階.

問 2 n 個の n 次元ベクトル $A_1, \dots, A_n \in \mathbb{Z}^n$ が与えられる. ただし $n \geq 68$ であり, ベクトル A_i の各成分の絶対値は高々 n^{100} である. これらの中から相異なる 68 個のベクトルを選んでその総和を零ベクトルにできるかどうかを $O(n^{34} \log n)$ 時間で判定するアルゴリズムを記述せよ. (10 点)

【解答】 与えられた入力を $A_1, \dots, A_n \in \mathbb{Z}^n$ とする. 以下の乱択アルゴリズムを考える:

1. ベクトル $r \in \mathbb{Z}^n$ を, 各成分を独立に $\{1, 2, \dots, 100 \cdot n^{68}\}$ の中から一様ランダムに選んで得られるランダムベクトルとする.
2. 各 $i = 1, \dots, n$ に対し, $a_i := A_i^\top r$ を計算する.
3. 集合 $S = \{a_1, \dots, a_n\}$ (多重集合とみなす) をランダムに 68 個の部分集合に分割し, $S = S_1 \sqcup \dots \sqcup S_{68}$ とする.
4. 各 S_i から一つずつ $s_i \in S_i$ を選び, $s_1 + \dots + s_{68} = 0$ となるような $s_1, \dots, s_{68}$ が存在するかどうかを $O(n^{34} \log n)$ 時間で判定し, もし存在するならば Yes を出力して終了する. これは, 全ての $s_1 \in S_1, \dots, s_{34} \in S_{34}$ に対し, $s_1 + \dots + s_{34}$ を格納したリスト L を作り, これをソートした上で, 各 $s_{35} \in S_{35}, \dots, s_{68} \in S_{68}$ に対し, $-(s_{35} + \dots + s_{68})$ が L の属するかどうかを二分探索して存在するかどうかを確認すれば $O(n^{34} \log n)$ 時間で判定できる.
5. ステップ 3 と 4 を十分大きな定数回, 具体的には $100 \cdot \frac{68^{68}}{68!}$ 回だけ繰り返し, 一度も Yes が出力されなければ No を出力して終了する.

この乱択アルゴリズムは確かに $O(n^{34} \log n)$ 時間で動作する. また, この乱択アルゴリズムは以下を満たす:

- 入力 $A_1, \dots, A_n$ の答えが Yes ならば, この乱択アルゴリズムは確率 0.99 で Yes を出力する.
- 入力 $A_1, \dots, A_n$ の答えが No ならば, この乱択アルゴリズムは確率 0.99 で No を出力する.

これら二つの主張を証明すれば, 上記の乱択アルゴリズムが正しく問題を解くことが示されたことになる.

一つ目の主張を証明する. 例えば $A_1 + \dots + A_{68} = 0$ となるならば, ステップ 2 で得られる $a_1, \dots, a_{68}$ は, $a_1 + \dots + a_{68} = 0$ となる. さらに, これら 68 個のベクトルは, ステップ 3 において, 確率 $\frac{68!}{68^{68}}$ で異なるグループに属し, ステップ 4 において正しく検知される. これを $100 \cdot \frac{68^{68}}{68!}$ 回繰り返すことで, どれか一回の反復においては正しく Yes が検知される確率は少なくとも 0.99 となる: 実際, 全ての反復で失敗する確率は

$$\left(1 - \frac{68!}{68^{68}}\right)^{100 \cdot \frac{68^{68}}{68!}} \leq e^{-100} \leq 0.01$$

である.

二つ目の主張を証明する. まず, 以下の補題を示す: 任意の非ゼロベクトル $u \in \mathbb{R}^n$ とステップ 2 で考えたランダムベクトル r に対し, 以下が成り立つ:

$$\Pr[u^\top r = 0] \leq \frac{1}{100 \cdot n^{68}}. \quad (1)$$

これを示す. r の成分を $r = (r_1, \dots, r_n)$ とする. u は非ゼロベクトルなので, 適当な非ゼロ成分を持つ. u の成分は入れ替えても確率は不変なので, $u_1 \neq 0$ としても一般性を失わない. このとき, 以下が成り立つ:

$$\Pr[u^\top r = 0] = \Pr\left[r_1 = -\frac{1}{u_1} \sum_{i=2}^n u_i r_i\right]$$

$r_2, \dots, r_n$ の何であつても, 右辺の確率の中身を満たす r_1 は高々一通りしか存在しない. 一方で r_1 は $1, \dots, 100 \cdot n^{68}$ の中から選ばれているため, 上記の確率は高々 $\frac{1}{100 \cdot n^{68}}$ となる (補題の証明終). なお, 別解として, 関数 $h: r \mapsto u^\top r$ を n 変数一次多項式とみなして Schwartz-Zippel の補題を適用する方法もある.

二つ目の主張の証明に戻る. 任意の $1 \leq i_1 < i_2 < \dots < i_{68} \leq n$ に対し, $A_{i_1} + \dots + A_{i_{68}}$ は非ゼロベクトルである. これを u として式 (1) を適用すると, ステップ 2 で選んだランダムベクトル r に対し, $a_1 + \dots + a_{68} = 0$ となる確率は高々 $\frac{1}{100 \cdot n^{68}}$ となる. 従って, ある $1 \leq i_1 < i_2 < \dots < i_{68} \leq n$ が存在して $A_{i_1} + \dots + A_{i_{68}} = 0$ となる確率は

$$\Pr\left[\bigvee_{1 \leq i_1 < i_2 < \dots < i_{68} \leq n} [A_{i_1} + \dots + A_{i_{68}} = 0]\right] \leq n^{68} \cdot \frac{1}{100 \cdot n^{68}} = 0.01$$

となる. 余事象を考えると, 確率 0.09 で, $(a_1, \dots, a_n)$ は 68SUM の No インスタンスとなる. すなわち, 全ての $1 \leq i_1 < i_2 < \dots < i_{68} \leq n$ に対し, $a_{i_1} + \dots + a_{i_{68}} \neq 0$ となる. このとき, ステップ 3 以降では常に No が出力されるため, 二つ目の主張が証明された.

採点基準. ひとまず, 「ランダムに分割して二分探索」「ランダムベクトルを考える」のそれぞれに 6 点と 4 点を配点する. これら二つの項目については必要に応じて部分点を付与する.

- 0 点, 2 点, 4 点, 6 点のどれか. 二分探索、ランダム分割のアイデアが出ていればそれぞれに 2 点. 解析も完璧で十分大きな定数回繰り返すところまで記述されていれば, この項目は 6 点満点とする.
- 0 点, 2 点, 4 点のどれか. 何でも良いのでランダムベクトルとの内積に関する言及があればひとまず 2 点加算. 完璧ならば 4 点満点.

例えば, 「ハッシュ関数を使う」など曖昧で具体的な記載がないがランダムに分割して二分探索を提示しており, ランダム分割や二分探索の解析が完璧な解答だと, それぞれの項目は 6 点と 0 点となり, 合計 6 点となる. 他の例として, ランダムベクトルとの内積までは完璧だが, 具体的に考えるランダムベクトルの中身が適切でない場合は 8 点とする. 他の解き方で合つてそうなものがあれば, 要相談.

問 3 全点对最短経路問題に対する Seidel 法 (第 4 回講義資料参照) において, 2-ホップグラフ $\tilde{G}$ とその二頂点 $u, v \in V$ を考ふ. $\tilde{G}$ 上の距離 $\text{dist}(u, v)$ が偶数のとき, v の全ての隣接頂点 i に対して $\text{dist}(u, i) \geq \text{dist}(u, v)$ が成り立つことを示せ. (10 点)

【解答】

解説. $\tilde{G}$ に含まれていて G に含まれない辺をショートカット辺と呼ぶことにする. $\tilde{G}$ 上の距離 $\text{dist}(u, v)$ が偶数のとき, $\tilde{G}$ 上の全ての最短 uv 路はショートカット辺のみからなる. 一方, もしも $\text{dist}(u, i) < \text{dist}(u, v)$ となるような i が v の隣接点の中に存在するならば, まず最短 ui 路をたどり, その後に辺 $\{i, v\}$ を通る uv 路を P とすると, その長さは

$$\widetilde{\text{dist}}(u, i) + 1 \leq \widetilde{\text{dist}}(u, v)$$

となる. すなわち P は最短 uv 路の一つである. しかし, これは全ての最短 uv 路がショートカット辺のみからなることに矛盾する.

採点基準. 解説以外にもいろんなバリエーションがあると思うが, ひとまず 0, 5, 10 点のどれかとする。

問 4 無向グラフ $G = (V, E)$ に含まれる路 $P = (v_0, \dots, v_\ell)$ は, ある $0 \leq i < j < \ell$ が存在して $(v_i, v_{i+1}) = (v_{j+1}, v_j)$ を満たすとき, バックトラックするという (図 2 を参照). n 頂点の無向グラフ G と二頂点 s, t が与えられたとき, G がバックトラックしない長さがちょうど 247 の st 路を含むかどうかを, $O(n^3)$ 時間で判定するアルゴリズムを記述せよ (入力 は隣接行列の形で与えられるとする). (10 点)

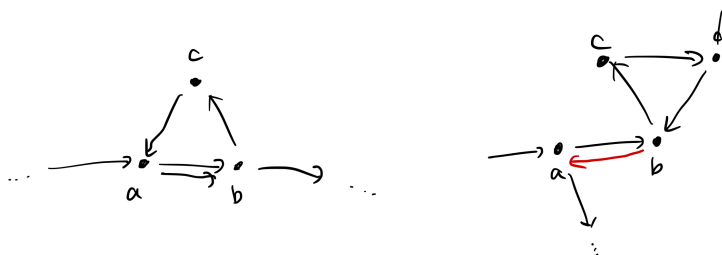


図 2: 右の路は辺 $\{a, b\}$ を両方の向きで通過しているためバックトラックする.

【解答】

解説. 与えられたグラフを $G = (V, E)$ とし, 二頂点を $s, t \in V$ とする. 以下の乱択アルゴリズムを考える:

1. 全ての辺 $e = \{u, v\} \in E$ に一様ランダムに向きを付けて (u, v) または (v, u) のどちらかを選ぶ. このようにして得られた有向グラフを $G' = (V, E')$ とする.
2. G' の有向隣接行列を B とする. すなわち, G' が辺 (u, v) 含むとき $B_{u,v} = 1$ であり, それ以外するとき $B_{u,v} = 0$ とする.
3. B^{247} を計算し, $B_{s,t}^{247} \geq 1$ を満たすならば, 「Yes」を出力して終了する.
4. ステップ 1 から 3 を十分大きな定数回, 具体的には $100 \cdot 2^{247}$ 回だけ繰り返し, 一度も Yes が出力されなければ No を出力して終了する.

このアルゴリズムは確かに $O(n^3)$ 時間で動作する. また, このアルゴリズムは以下を満たす:

- 入力 G の答えが Yes ならば, このアルゴリズムは確率 0.99 で Yes を出力する.
- 入力 G の答えが No ならば, このアルゴリズムは確率 1 で No を出力する.

これら二つの主張を証明すれば, 上記のアルゴリズムが正しく問題を解くことが示されたことになる.

一つ目の主張を証明する. グラフ G がバックトラックしない長さがちょうど 247 の st 路 $P = (v_0, \dots, v_{247})$ を含むとする. このとき, 確率 2^{-247} で, ステップ 1 で得られる有向 G' においてこの路を構成する全ての辺は $v_i \rightarrow v_{i+1}$ の向きになる. すなわち, 少なくとも確率 2^{-247} で $B_{v_i, v_{i+1}}^{247} = 1$ となるため, アルゴリズムは Yes を出力する.

二つ目の主張を証明する. このとき, 長さ 247 の任意の st 路 $P = (v_0, \dots, v_{247})$ に対し, ある辺 $\{u, v\} \in E$ が存在して, P はこの辺を $u \rightarrow v$ と $v \rightarrow u$ の両方の向きで通過する. 従って, どのように向きをつけても P は有向グラフ上の路になりえない. すなわち, 確率 1 で $B_{v_i, v_{i+1}}^{247} = 0$ となるため, アルゴリズムは No を出力する.

合ってそうだが実は間違ってる解答. バックトラックしない長さ k の uv 路の本数を (u, v) 成分に格納した行列 L_k に対し, 漸化式を与える.

まず, $k = 0$ のときは $L_k = I$ (単位行列) になる. $k = 1$ のときは $L_k = A$ (A はグラフ G の隣接行列) になる.

$k \geq 2$ に対して L_k を考える. まず, $L_{k-1} \cdot A$ に着目すると, その (u, v) 成分は, 長さ k の uv 路 ($u = u_0, \dots, u_{k-1} = v$) であって,

部分路 $(u_0, \dots, u_{k-2})$ がバックトラックしないものの個数 (B)

に等しい. 従って, $L_k[u][v]$ を求めるには, $(L_{k-1} \cdot A)[u][v]$ から, (B) を満たし, かつ最後の辺 $\{u_{k-1}, u_k\}$ でバックトラックするような uv 路を除けばよい. 最後の辺でバックトラックが発生するということは $u_{k-2} = u_k$ となっている. つまり, u から v まで長さ $k-2$ のバックトラックしない路を構成し, そこから v の任意に隣接頂点 $w \in N(v)$ を選んで辺 $\{v, w\}$ を双方向に辿る路を後ろに追加して得られる. このような路の本数は $L_{k-2}[u][v] \cdot \deg(v)$ に等しい. よって漸化式

$$L_k[u][v] = (L_{k-1} \cdot A)[u][v] - \deg(v) \cdot L_{k-2}[u][v]$$

が成り立つ. これは, 各頂点の次数を対角成分に並べて得られる行列を D とすると

$$L_k = L_{k-1} \cdot A - D \cdot L_{k-2}$$

と書ける. これを $k = 0, 1, \dots, 247$ に対して順に計算し, $L_{247}[s][t] \geq 1$ かどうかを判定することで, グラフ G がバックトラックしない長さがちょうど 247 の st 路を含むかどうかを $O(n^3)$ 時間で判定できる.

採点基準. アルゴリズムのポイントは以下の通りである:

- 辺にランダムな向きをつけて有向グラフに変換しているか (6 点).
- 隣接行列の累乗を計算して到達可能性を判定しているか (2 点).
- 解析 (2^{-247} が書いてある) が OK, かつ繰り返して成功確率を増加させることに言及があるか (2 点).

なお, 有向グラフ版の隣接行列の累乗が有向路の本数に対応するという事実は証明なしに用いてよいものとする.

また, 上記の「合ってそうだが実は間違ってる解答」に関しては、以前に学生からの DM で私が誤って「正しい」と言ってしまう、これで 0 点扱いにするのは流石に理不尽なので、同じ漸化式を立てて一見正しそうな「導出」してる場合（本当は導出できないが）は救済として 8 点、漸化式は違うが同じような考え方を論理的に述べている場合は 6 点としてください。適当に漸化式を出すだけだと 0 点です。

問 5 第 6 回課題の問 1 で考えた主問題 (P) とその双対問題 (D) を考える:

$$(P) \quad \begin{array}{ll} \text{minimize} & c^\top x \\ \text{s.t.} & Ax \geq b \\ & x \geq 0 \end{array}$$

$$(D) \quad \begin{array}{ll} \text{maximize} & y^\top b \\ \text{s.t.} & y^\top A \leq c^\top \\ & y \geq 0 \end{array}$$

第 6 回課題では非負制約 $x \geq 0$ を関数 $\mathcal{L}^*(y) := \inf_{x \geq 0} \{c^\top x + y^\top(b - Ax)\}$ に組み込んで (D) を導出した. 今回は非負制約を行列を用いて書き換えた問題 (P') について, 第 6 回講義資料と同様の方法で双対問題を導出せよ:

$$(P') \quad \begin{array}{ll} \text{minimize} & c^\top x \\ \text{s.t.} & \begin{bmatrix} A \\ I \end{bmatrix} x \geq \begin{bmatrix} b \\ 0 \end{bmatrix} \end{array}$$

ただし, $I \in \mathbb{R}^{n \times n}$ は単位行列である. また, $Ax \geq b$ に対応する双対変数を $y \in \mathbb{R}^m$, $Ix \geq 0$ に対応する双対変数を $z \in \mathbb{R}^n$ とせよ. さらに, この導出した双対問題を (D') としたとき, (D) と (D') が等価であることを示せ. つまり, 一方の最適解を用いて他方の最適解を構成できることを示せ. (10 点)

【解答】

解説. (P') の双対問題は以下のように導出できる: まず, 行列 $A' \in \mathbb{R}^{(m+n) \times n}$ を $A' := \begin{bmatrix} A \\ I \end{bmatrix}$ と定義する. 同様に, ベクトル $b' \in \mathbb{R}^{m+n}$ を $b' := \begin{bmatrix} b \\ 0 \end{bmatrix}$ と定義する. $Ax \geq b$ に対応する双対変数を $y \in \mathbb{R}^m$, $Ix \geq 0$ に対応する双対変数を $z \in \mathbb{R}^n$ とする (ただし, $y, z \geq 0$). 関数

$$\mathcal{L}^*(y, z) := \inf_x \{c^\top x + y^\top(b - Ax) - z^\top x\} \quad (2)$$

を考える. この関数を式変形すると

$$\begin{aligned} \mathcal{L}^*(y, z) &= y^\top b + \inf_x \{(c^\top - y^\top A - z^\top)x\} \\ &= \begin{cases} y^\top b & \text{if } c^\top - y^\top A - z^\top = 0 \\ -\infty & \text{otherwise} \end{cases} \end{aligned} \quad (3)$$

となる. 従って双対問題 (D') は以下ようになる:

$$(D') \quad \begin{array}{ll} \text{maximize} & y^\top b \\ \text{s.t.} & y^\top A + z^\top = c^\top \\ & y \geq 0 \\ & z \geq 0 \end{array}$$

次に (D) と (D') が等価であることを示す. (D) の最適解を $y^* \in \mathbb{R}^m$ とする. このとき, $(z^*)^\top = c^\top - y^{*\top} A$ とすれば, (y^*, z^*) は (D') の最適解となる. 一方で, (D') の最適解 (y^*, z^*) に対し, y^* は (D) の最適解となる.

採点基準. 以下の五つのポイントそれぞれに 2 点ずつ. なお, 簡単な式変形で本質的に等価なものが得られているなら丸扱いとする.

- 関数 (2) を定義しているか (2 点).
- この $\mathcal{L}^*(y, z)$ を, $\inf$ を除去した形の式 (3) で表しているか (2 点).
- そこから正しく (D') を導出しているか (2 点).
- (D') の最適解 (y^*, z^*) を用いて (D) の最適解を正しく構成できているか (2 点). 証明はなくても良い.
- (D) の最適解 y^* を用いて (D') の最適解を正しく構成できているか (2 点). 証明はなくても良い.